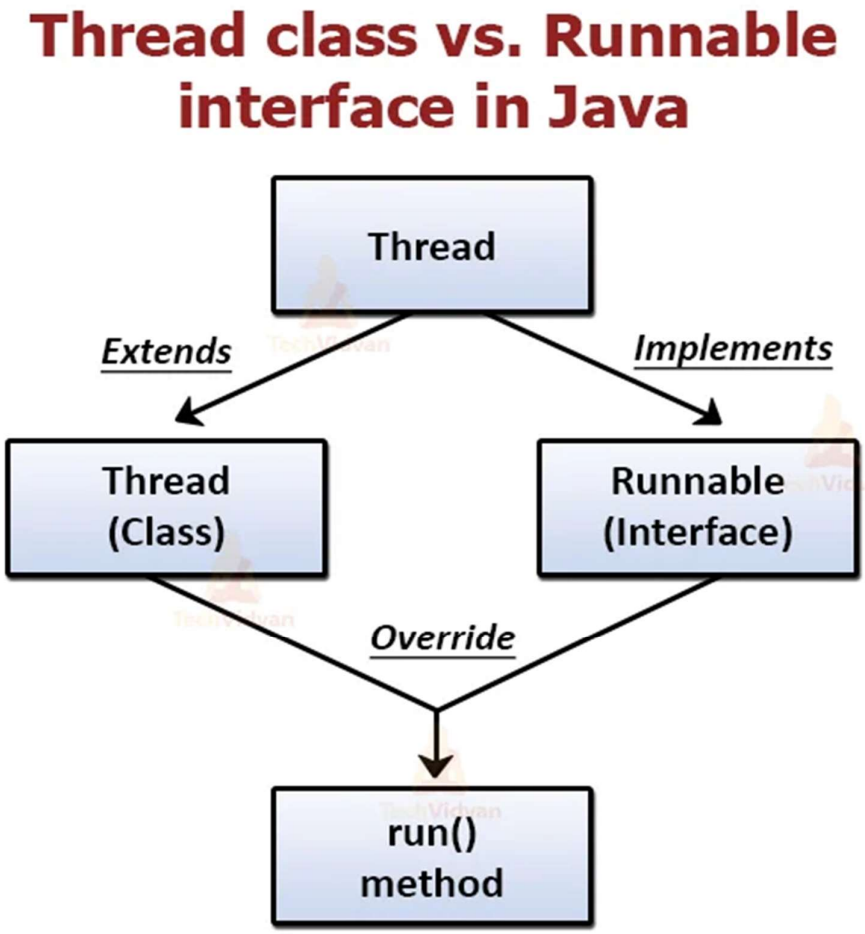
Create Thread in Java

Two Ways of create threads :-



- Threads can be created in two ways i.e. by **implementing java. lang.** Runnable interface or **extending java. lang.**
- The Java Virtual Machine allows an application to **have multiple threads** of **execution running concurrently.**
- One is to declare a class to be a **subclass of Thread**; The other way to create a thread is to declare a class that **implements the Runnable interface.**

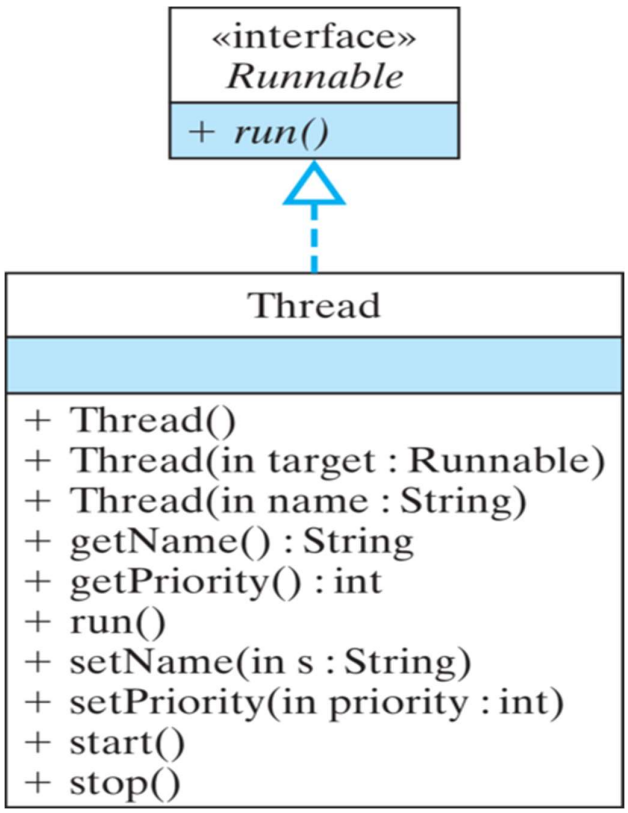
Thread Class

What is thread class :-

- The Thread class in Java is used to create and control threads the **smallest unit of a program** that can run **independently**.
- Defined in package : **java.lang**

Purpose :-

- It allows you to create and manage **multiple threads for multitasking** (executing multiple parts of a **program simultaneously**).
- You can create a thread by **extending the Thread class** and **overriding the run()** method.



Implementation of thread class :-

Input :-

```java
package Multi_Threading;

public class multithreading_2 extends Thread{

    public void run()
    {
        System.out.println("Running Thread");
        try {
            Thread.sleep(5000);
        } catch (InterruptedException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }
        System.out.println("Executed....!");
    }

    public static void main(String[] args) {
        multithreading_2 th1 = new multithreading_2();
        System.out.println(th1.getState()); // New
        th1.start();
        System.out.println(th1.getState()); // Runnable
        System.out.println("is alive for first time : "+th1.isAlive());

        System.out.println("is alive for second time : "+th1.isAlive());
        System.out.println(th1.getState()); // Terminated
    }
}
```

Output :-

```
NEW
RUNNABLE
Running Thread
is alive for first time : true
is alive for second time : true
TIMED_WAITING
Executed....!
```

Thread Life Cycle

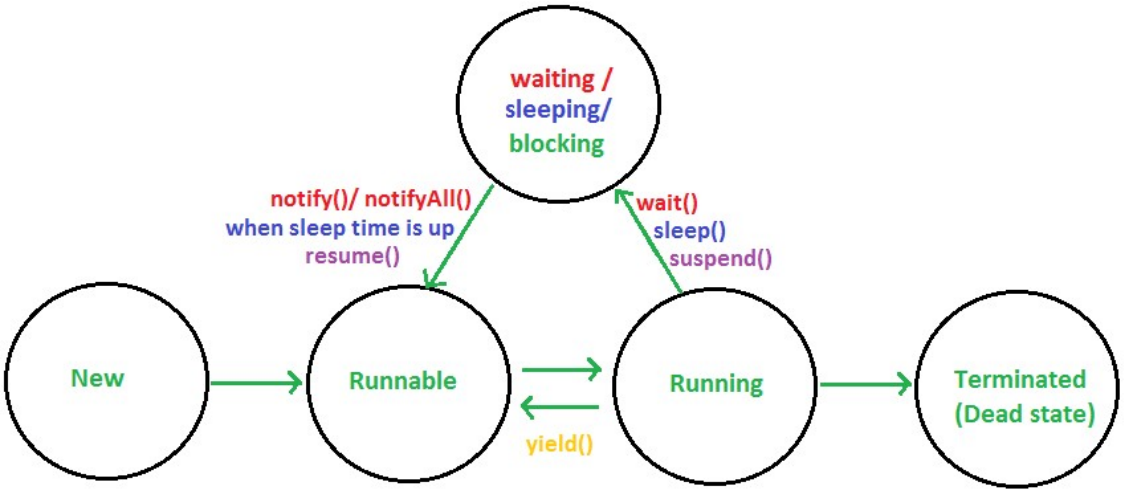


Fig. THREAD STATES

What is thread life cycle :-

- The **Java Thread Lifecycle** shows the different stages a thread goes through — from creation to ending.
- These stages are defined in the **Thread.State enum** in Java.
- Knowing these states helps in **understanding and controlling** how multiple threads run at the same time in a program.

Description of life cycle :-

- **New:** A thread is created but not yet started using start().
- **Runnable:** After calling start(), the thread becomes ready to run and waits for CPU time.
- **Running:** The thread is actively executing its run() method.
- **Waiting/Sleeping/Blocking:** The thread is temporarily inactive due to wait(), sleep(), or suspend() methods.**Terminated (Dead):** The thread finishes execution or stops due to an error.

Runnable Interface

What is runnable interface :-

- The Runnable **interface** in Java is used to define a task that can be executed by a thread.
- Defined in package : **java.lang package.**

Purpose :-

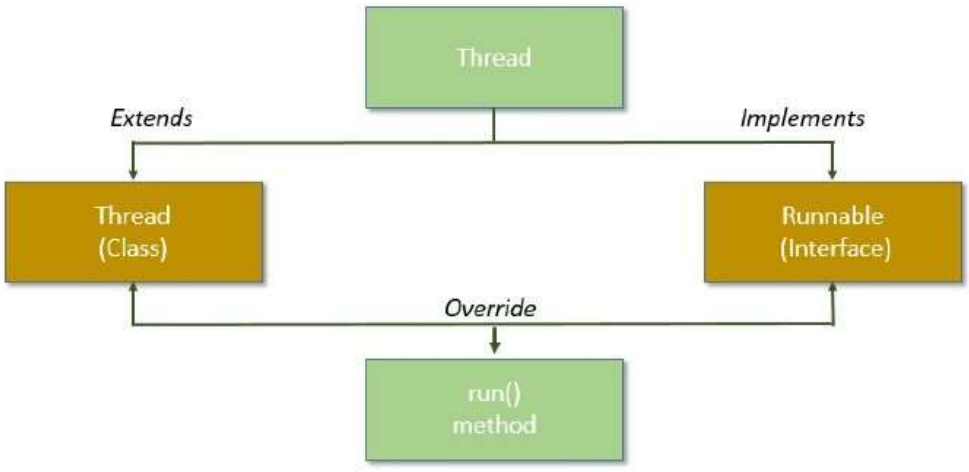
- It represents a task that can run in a separate thread — useful when you want to achieve **multithreading** without extending the Thread class.

How to Use :-

- Implement the Runnable interface.
- Override its single method run().
- Pass the object of that class to a Thread object, then call start().

Methods :-

- void run() → contains the code that will be executed by the thread.



Implementation of Runnable Interface :-

Input :-

```java
package Multi_Threading;

public class multithreading_3 implements Runnable{

    public void run()
    {
        System.out.println("Running Task");
    }

    public static void main(String[] args) {

        multithreading_2 th2 = new multithreading_2(); //New

        Thread thr2 = new Thread(th2); // Creating object of thread class
        thr2.start();
    }
}
```

Output :-

```
Running Thread
```